

Lecture 05

The Shopkeeper's Ledger

Git for code, and what belongs in a repository



a copy says nothing about what changed, when, or why

Recap of Lecture 04 in three points. Six models, one honest comparison, on the same split and the same metric. The simplest model won on our data, and we published that. k means finds groups when nobody labelled anything.

That winning model came from *code*. The code lives in one folder, on one laptop, with no history at all. Today we fix that, and it is the habit every later lecture assumes.

Contents

How this handout maps to the approved session plan	4
I Git for Code	5
1 A folder you have already seen	5
2 The shopkeeper's ledger	5
3 The three places your change passes through	6
4 Making one ledger entry	6
5 Reading the ledger back	6
6 What a good entry says	6
7 What a commit actually contains	6
8 Common errors when making entries	7
II Working Together	8
9 Two cooks, one pot	8
10 Branches: a separate stove for experiments	8
11 The pull request: taste before serving	9
12 When the push is rejected	9
13 When both cooks touched the same line	9
14 Common errors when working together	10
III What Belongs in the Ledger	11
15 The one line change that looked like four hundred	11
16 What belongs in the ledger	11
17 Teaching Git what to ignore	12
18 Two things the ignore file cannot undo	12

19 Common errors with notebooks and data	12
 IV Consolidation	 13
20 What our project gains today	13
21 Assignment questions, as released	13
22 Check your understanding	14
23 Vocabulary introduced today	14
 V The Week Ahead	 16
24 Seven days, about seven hours	16
 VI Resource Library	 18
25 For this lecture specifically	18
26 Carried forward	18

List of Figures

1	Copies are not history. A copy records the state of a file at some unknown moment; a history records the sequence of decisions that produced it.	5
2	The four places, and the three commands that move a change between them. Most confusion in the first fortnight comes from not knowing which of these a file is currently sitting in.	6
3	The anatomy of one commit. The parent pointer is what turns a pile of snapshots into a history, and the hash is what makes a reference to that history trustworthy.	7
4	The chain of commits, and the artifact that references one of them. This single arrow is the foundation of every lineage claim you will make for the rest of the course.	7
5	One shared copy, many laptops. The remote is not a backup; it is the agreed point of truth that every laptop reconciles against.	8
6	A branch and its pull request. Note what the diagram makes obvious: at no point was the working recipe on the main line unavailable to anybody else.	8
7	The rejected push, and why it happens. The remote moved on after Ravi last synchronised, so his push would have to overwrite somebody else's work to succeed.	9
8	The three markers, and what to do with them. The middle divider separates the two versions; everything between the markers is yours to resolve.	10
9	Why notebook diffs are unreviewable by default. Nothing is wrong with your edit; the file format simply stores far more than the code you wrote.	11
10	The rule of thumb that decides every case you will meet: commit what a human wrote, and keep out what a machine produced.	11
11	What today added. Not a new capability on the arc, but the foundation the whole arc rests on: tracking, containers and automated delivery all assume a repository with a history.	13

How this handout maps to the approved session plan

Topic in this handout	Session plan location	Note
Version control philosophy: Git for code	Session 9	The core of the session
Collaborative workflows: branches, pushes, pull requests	Session 9	Extended here with the rejected push and why force pushing is destructive
Notebooks versus scripts, and what belongs in a repository	Session 9	Extended with the professional fix for notebook diffs, and with secrets
Data versioning	Session 10	Deliberately not here. Today ends by naming the gap that the next lecture fills

One factual correction to the released deck

The recap frame of the released L05 slides states that the random forest won in Lecture 04. It did not. The measured table in Lecture 04 shows linear regression at 1.92 minutes and the random forest at 2.43 on the same test set, and the deck for that lecture says so explicitly.

The forest is the model the project *carries forward* from Lecture 11, for a stated engineering reason, and that is a different claim. The distinction matters: the whole point of Lecture 04 was that we measured rather than assumed, and a recap that reverses the result undoes the lesson. Worth correcting in the slides before delivery.

The taught hour, minute by minute

0 to 4	Recap, then Section 1. Show the folder of four copies and ask which one shipped
4 to 12	Sections 2 and 3, the ledger and the three places a change passes through
12 to 20	Sections 4 to 6, making an entry, reading the history, and what a good message says
20 to 28	Section 7, the anatomy of a commit. This is where the lineage argument lands
28 to 38	Sections 9 to 11, the shared copy, branches and pull requests
38 to 46	Sections 12 and 13, the rejected push and the merge conflict
46 to 56	Sections 15 to 17, notebooks, what belongs in the ledger, and the ignore file
56 to 60	Section 19, what the project gains, and the closing line

PART I

Git for Code

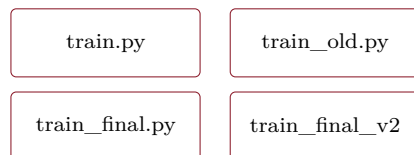
1 A folder you have already seen

A student's project folder, the night before the demonstration:

```
train.py
train_old.py
train_final.py
train_final_v2_FIXED.py
```

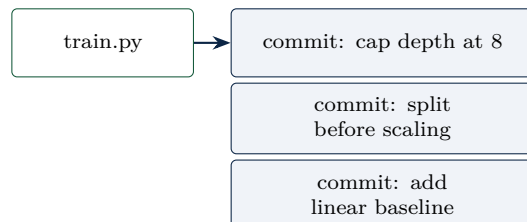
The deployed model was trained by *one* of these. Which one? Nobody knows.

FOUR COPIES



Four files, no order, no dates, no reasons. Nothing records which one produced the model that shipped.

ONE FILE, ONE HISTORY



One file. Every earlier state still recoverable, each with a reason attached.

Figure 1: Copies are not history. A copy records the state of a file at some unknown moment; a history records the sequence of decisions that produced it.

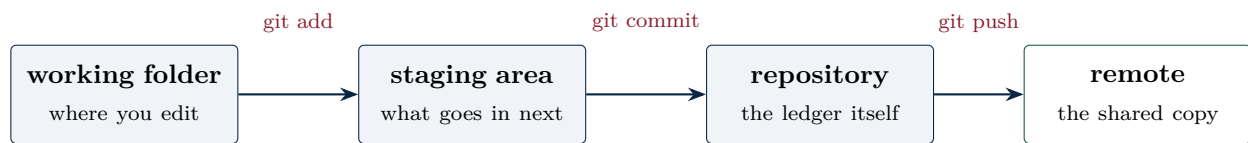
2 The shopkeeper's ledger

A shopkeeper never keeps four copies of the shop. He keeps one shop and one ledger. Every transaction is written down as it happens, nothing is ever erased, corrections are new entries, and anyone can audit the whole story years later.

Version control

A version control system is a ledger for files. It records every change, who made it, when, and why. **Git** is the version control system the whole industry uses.

3 The three places your change passes through



A commit is one ledger entry: a named snapshot. Staging is what lets you choose which changes belong in that entry rather than dumping everything you happen to have touched.

Figure 2: The four places, and the three commands that move a change between them. Most confusion in the first fortnight comes from not knowing which of these a file is currently sitting in.

4 Making one ledger entry

```
git status          # what changed since the last entry?
git add train.py    # choose what the entry contains
git commit -m "Split data before scaling to avoid leakage"
```

`status` costs nothing, so run it constantly. `add` stages the file and records nothing yet. `commit` writes the entry, stamped with your name and the time.

5 Reading the ledger back

Six entries into the delivery time project, the history reads:

```
git log --oneline
d6bbbcd Ignore data files and notebook checkpoints
c20ffe5 Cap max_depth at 8 to stop overfitting
6bc7874 Add random forest; compare against baseline
b1b24c5 Split data before scaling to avoid leakage
9b54b2e Add linear baseline for delivery-time prediction
4fca8f0 Add project brief and data dictionary
```

Newest first. Each code such as `d6bbbcd` names one snapshot forever. This is the story of the project, readable in ten seconds, and it is the thing a folder of copies can never give you.

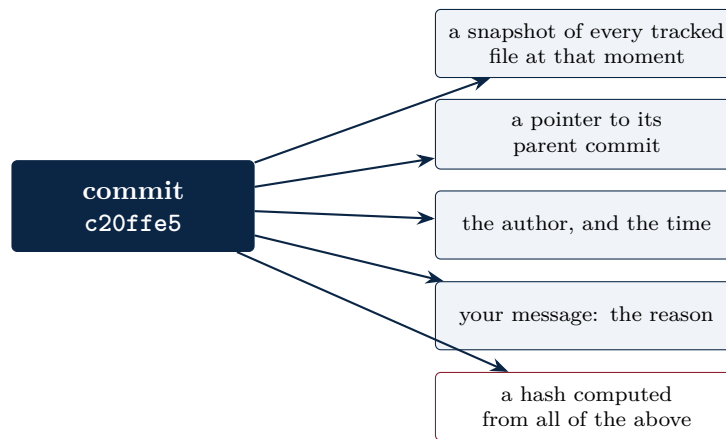
6 What a good entry says

Weak message	Strong message
update	Split data before scaling to avoid leakage
fix	Cap max_depth at 8 to stop overfitting
final changes	Add random forest; compare against baseline

The difference is not politeness. The diff already shows *what* changed, and it will still show it in five years. Only you can record *why*, and only in the moment you made the change.

7 What a commit actually contains

A commit is not a copy of your changes. It is a small record with five parts, and understanding those five parts is what turns Git from a set of memorised commands into something you can reason about.



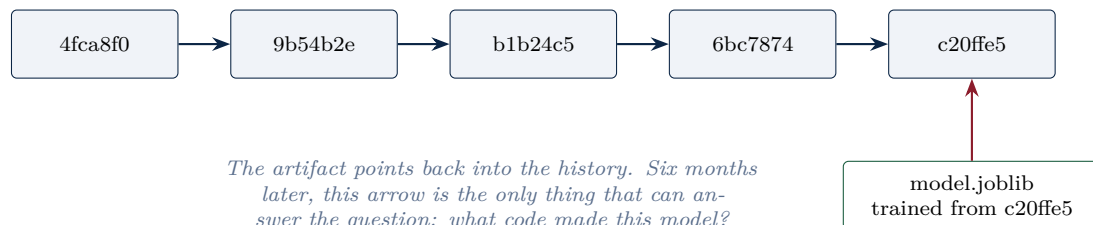
Because the hash is computed from the content, the same hash always means the same code. That is why a model artifact can record a commit hash and have it mean something permanent.

Figure 3: The anatomy of one commit. The parent pointer is what turns a pile of snapshots into a history, and the hash is what makes a reference to that history trustworthy.

Why this matters for the rest of the course, and not only for Git

In Lecture 04 we listed what has to travel with a model artifact: the input signature, the library versions, the seed and dataset version, and the score. Add one more, and it is the important one.

The commit hash that produced it. A model that records `c20ffe5` can be traced to exactly the code that built it, forever, by anybody. Without it, a model file is an orphan. From Lecture 11 the tracking server records this automatically for every run, and it can only do so because you committed.



The artifact points back into the history. Six months later, this arrow is the only thing that can answer the question: what code made this model?

Figure 4: The chain of commits, and the artifact that references one of them. This single arrow is the foundation of every lineage claim you will make for the rest of the course.

8 Common errors when making entries

The error	What actually happens
Editing, then committing without staging	Git replies that nothing was added to commit. Stage first, then commit
Running <code>git add .</code> without looking	It stages everything, including data files and credentials
Messages like <code>final2</code>	In two weeks they mean nothing, even to you
Committing once a week	Small, frequent entries. A ledger with one giant entry is not a ledger
Committing a credential “temporarily”	See Section 18. Deleting it later does not remove it from the history

PART II

Working Together

9 Two cooks, one pot

In the second practical you worked in pairs. Imagine doing that by email. Asha sends `train.py` to Ravi. Ravi edits and sends it back, but Asha has edited meanwhile. Someone now merges two files by hand, line by line.

The last person to paste, wins. Everyone else's work is lost, silently, with no record that it ever existed.

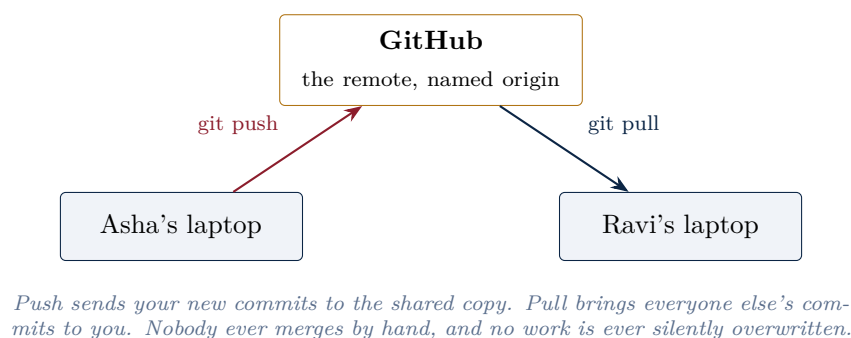
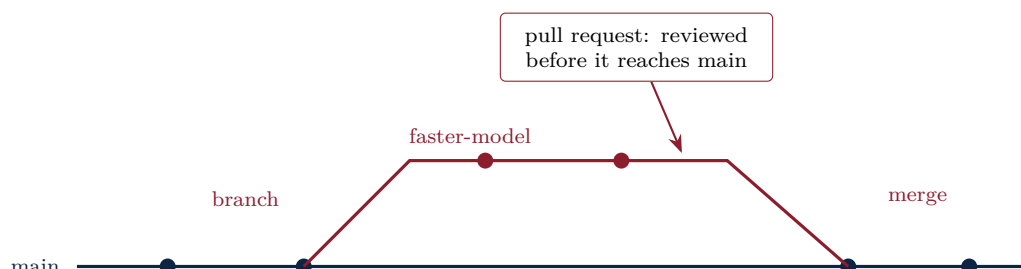


Figure 5: One shared copy, many laptops. The remote is not a backup; it is the agreed point of truth that every laptop reconciles against.

10 Branches: a separate stove for experiments

A branch lets you develop a change in a contained area of the repository without touching `main`. The default branch in a new repository is named `main`, and the working rule is that it must always run.

```
git switch -c faster-model    # new branch, and move onto it
git add train.py
git commit -m "Try 300 trees to cut MAE further"
git push -u origin faster-model
```



The main line never stops working. The experiment happens beside it, is reviewed, and only then joins. If the experiment fails, the branch is deleted and nothing was risked.

Figure 6: A branch and its pull request. Note what the diagram makes obvious: at no point was the working recipe on the main line unavailable to anybody else.

11 The pull request: taste before serving

A pull request is a proposal to merge code changes, discussed and reviewed before merging. The Files changed tab shows reviewers the exact difference, line by line.

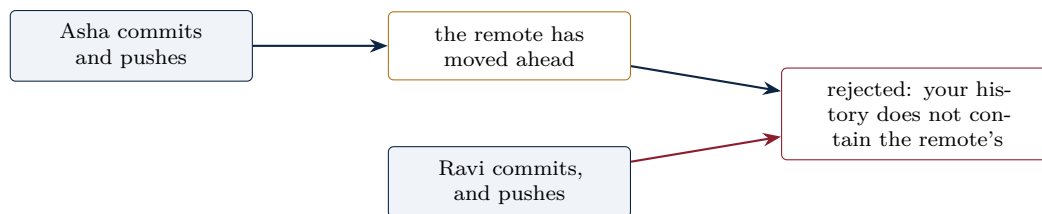
Why review is an engineering control, not a courtesy

Two problems disappear the moment a second person reads the difference before it reaches the main line. First, mistakes are caught while they are cheap, when the author still remembers the reasoning. Second, knowledge stops being trapped in one head, which is the failure that ends projects when somebody leaves.

There is a third, specific to this course: a pull request is where the automated checks run. From Lecture 20, a pipeline will run the tests on every pull request and block a merge that breaks them. None of that machinery exists without this habit.

12 When the push is rejected

Asha pushes to main. Ravi, who has not pulled, now pushes his own commit and Git refuses.



This is not an error in your work. Git is refusing to discard commits it can see and you cannot, which is the single most useful thing it ever does for a team.

Figure 7: The rejected push, and why it happens. The remote moved on after Ravi last synchronised, so his push would have to overwrite somebody else's work to succeed.

The fix is three commands, in this order:

```
git pull      # bring the remote's commits into your history
              # resolve any conflict here, then commit
git push      # now your history contains theirs, and it is accepted
```

Never solve this with a forced push

`git push -force` makes the rejection go away by replacing the remote's history with yours. It works, in the sense that the error disappears, and it deletes your teammate's commits from the shared copy. They will discover this when their work vanishes.

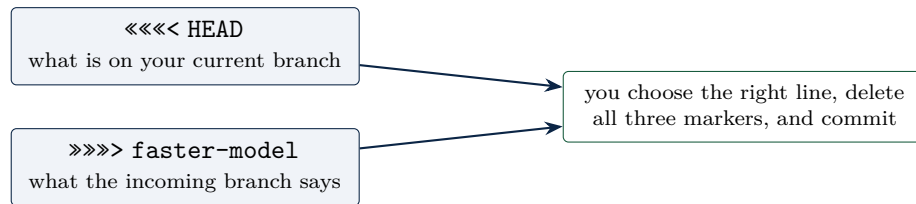
If you find yourself reaching for it, stop and pull instead. On a shared branch there is no situation in this course where forcing is the right answer.

13 When both cooks touched the same line

Git merges most changes silently. When two commits change the *same line*, it stops and shows you both.

```
<<<<<<< HEAD
n_estimators = 150
=====
```

```
n_estimators = 300
>>>>> faster-model
```



A merge conflict is normal, not a crash. Git never guesses which line is correct, because guessing would silently discard somebody's intent. It makes a human decide, which is the right design.

Figure 8: The three markers, and what to do with them. The middle divider separates the two versions; everything between the markers is yours to resolve.

14 Common errors when working together

The error	The correction
Committing straight to <code>main</code>	Experiments belong on branches. The main line must always run
Pushing before pulling	If the remote moved on, your push is rejected. Pull, resolve, then push
Forcing the push to make it work	You just erased your partner's commits from the shared copy
A pull request with forty changed files	Nobody can review it, so nobody does. Small branches, small requests
Leaving a branch open for three weeks	The longer it lives, the harder the eventual merge

PART III

What Belongs in the Ledger

15 The one line change that looked like four hundred

Asha changes one number in a notebook and commits it. The review shows hundreds of changed lines.

What a notebook file really is

A notebook is one large JSON file. The code, the outputs, the charts and the execution counters are all stored inside it. Re running a cell changes the file even if you edited nothing at all.

WHAT YOU CHANGED

```
one number
n_estimators =
150 became 300
```

A single meaningful edit, which is what you want a reviewer to see.



WHAT GIT SEES

```
hundreds of changed lines: outputs,
chart data, execution counters
```

The real change drowns in noise, and review becomes impossible.

Figure 9: Why notebook diffs are unreviewable by default. Nothing is wrong with your edit; the file format simply stores far more than the code you wrote.

The professional fix, beyond remembering to clear outputs

Clearing outputs by hand works until the day somebody forgets, which is every team, eventually. The tool that removes the human step is **nbstripout**. Installed as a Git filter, it strips outputs from what Git sees while leaving your working copy intact, so you keep your charts on screen and the repository never records them. It also ships as a pre commit hook and as a reusable action that can verify notebooks are stripped before a merge.

pip install nbstripout, then nbstripout -install inside the repository. <https://github.com/kynan/nbstripout>

16 What belongs in the ledger

COMMIT IT

```
scripts such as train.py
notebooks, with outputs cleared
requirements.txt
the README, and configuration files
```

Things a human wrote, and things needed to rebuild everything else.

KEEP IT OUT

```
outputs and charts
.ipynb_checkpoints/
the dataset, delivery_times.csv
trained models, and .venv/
```

Things a machine produced, which can be produced again.

Figure 10: The rule of thumb that decides every case you will meet: commit what a human wrote, and keep out what a machine produced.

17 Teaching Git what to ignore

A file named `.gitignore`, committed at the root of the repository:

```
data/           # the dataset lives elsewhere
.ipynb_checkpoints/ # Jupyter's auto saves
*.pkl          # trained models are outputs
.venv/         # your environment, not your code
.env           # credentials, never
```

Ignored files vanish from `git status`, which means no more accidental disasters from staging everything at once.

18 Two things the ignore file cannot undo

If you already committed the dataset

Adding it to `.gitignore` afterwards stops future commits from tracking it, and does nothing about the copy already in the history. Git keeps every version of every file forever, by design. The repository stays large, every clone downloads it, and removing it properly means rewriting the history, which every collaborator then has to deal with.

This is not a reason to panic. It is a reason to write the ignore file *first*, before the first commit, which takes about a minute.

If you already committed a credential

Treat it as compromised, immediately. Deleting the line in a later commit does not remove it: the earlier commit still contains it, and anybody with the repository can read it. The only correct response is to revoke and reissue the key, and then clean the history.

This is why `git add .` without looking is a genuine hazard rather than a style complaint, and it is why the personal access token you created for this course should never appear in any file you commit.

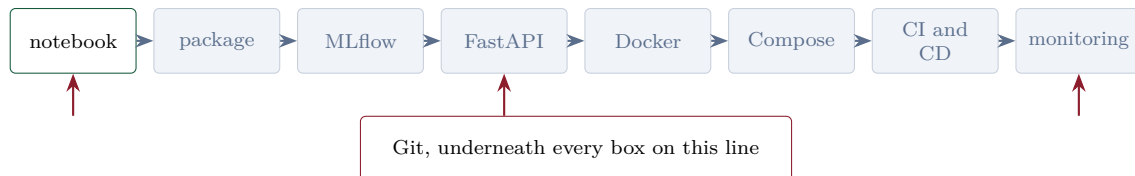
19 Common errors with notebooks and data

The error	The correction
Committing outputs	Every re run creates a false difference. Clear outputs, or install the filter
Committing <code>.ipynb_checkpoints/</code>	Automatic saves are noise. Ignore the folder
Committing the dataset just this once	Git keeps every version forever, and the repository balloons
So where does the data live?	Git alone cannot version large data. That gap is exactly the next lecture

PART IV

Consolidation

20 What our project gains today



Our project's code now has a history and a shared home. Every model we train from here can be traced back to the exact code that produced it, and every later stop on this arc reads from that history.

Figure 11: What today added. Not a new capability on the arc, but the foundation the whole arc rests on: tracking, containers and automated delivery all assume a repository with a history.

21 Assignment questions, as released

Set 1 • Git for code • CO1

Q1 (Understand). Define version control in your own words. Then give two questions a commit history can answer that a folder of copies named `train_final_v2.py` cannot.

Q2 (Apply). A student edits `train.py`, then runs `git commit -m "update"` and Git replies that nothing was added to commit. Explain what was skipped, give the two commands that fix it, and rewrite the message so it says *why*.

Q3 (Build). In your delivery time project repository from the first practical, make two commits: one improving the README, one changing a training setting. Each message must state a reason. Paste the output of `git log --oneline` showing both.

Set 2 • Collaborative workflows • CO1

Q1 (Understand). Explain what a branch and a pull request are, and give two concrete problems a team avoids by reviewing a pull request before merging into `main`.

Q2 (Apply). Asha and Ravi both cloned the same repository. Asha pushes a commit to `main`. Ravi, who has not pulled, now pushes his own commit. Predict what Git tells Ravi, and list the commands he must run, in order, to get his work onto GitHub.

Q3 (Build). With your partner from the second practical, on your delivery time repository: create a branch, commit one change to the training script with a message that says why, open a pull request, have your partner leave one review comment, then merge. Submit the pull request's address.

Set 3 • Notebooks versus scripts • CO1

Q1 (Understand). Explain why changing one number in a notebook can appear as hundreds of changed lines in review, and state the habit that prevents it. Then give two examples each of files that belong in Git and files that do not.

Q2 (Apply). A repository folder contains `train.py`, `delivery_times.csv`, `model.pkl`,

explore.ipynb and .ipynb_checkpoints/. Write the .gitignore you would commit, and state which files remain visible to `git status` afterwards.

Q3 (Build). In your delivery time repository: add that .gitignore, clear your notebook's outputs, and commit both with a message saying why. Paste the `git status` output from before and after, to show the ignored files disappearing.

22 Check your understanding

1. Give two questions a commit history answers that a folder of copies cannot.
2. Name the four places a change passes through, and the command that moves it between each pair.
3. What does staging let you do that committing everything at once does not?
4. List the five things a commit contains.
5. Why is a commit hash a trustworthy reference, and what does that buy a model artifact?
6. Rewrite `fix` as a message that would still be useful in a year.
7. What must always be true of the main line, and what follows from that for your experiments?
8. Ravi's push is rejected. Explain in one sentence what Git is protecting, and give the fix in order.
9. Why is forcing a push the wrong answer on a shared branch?
10. A merge conflict appears. Is something broken? What do you do with the three markers?
11. Why does changing one number in a notebook produce a huge difference, and what removes the human step from the fix?
12. You committed the dataset last week and have now added it to the ignore file. Is the repository small again? Explain.
13. State the rule of thumb for what belongs in a repository, and apply it to a trained model file.

23 Vocabulary introduced today

Term	Meaning
Version control system	A ledger for files: every change, who made it, when, and why
Repository	A project folder together with its complete history
Working folder	Where you edit. Changes here are recorded nowhere yet
Staging area	What your next commit will contain
Commit	One entry in the ledger: a snapshot, a parent, an author, a time, a message and a hash
Commit hash	The identifier computed from a commit's content, which names that state forever
Remote	The shared copy, usually on GitHub, conventionally named origin
Push and pull	Sending your commits to the remote, and bringing others' commits to you
Branch	A contained line of work that leaves the main line untouched
Pull request	A proposal to merge, reviewed before it is accepted

Term	Meaning
Merge conflict	Two commits changed the same line, so a human must choose
Fast forward	A push that only adds commits on top of what the remote already has
<code>.gitignore</code>	The committed list of paths Git should not track
<code>nbstripout</code>	A Git filter that strips notebook outputs from what Git records

PART V

The Week Ahead

24 Seven days, about seven hours

Day 1 • The basics, properly • 60 minutes

Read. Pro Git, chapter 2, Git Basics. Free and complete. <https://git-scm.com/book/en/v2>

Produce. Run `git log -stat` on your own repository and write two sentences on what the extra column tells you that `-oneline` does not.

Day 2 • Branching, until it is boring • 75 minutes

Read. Pro Git, chapter 3, Git Branching. The section on branching and merging in a workflow is the one that matters.

Produce. Create three branches, make one commit on each, merge two and delete the third. The aim is that branching stops feeling risky.

Day 3 • The two official pages • 45 minutes

Read. GitHub Docs on branches and on pull requests, the two pages the lecture quotes.

<https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-branches>

<https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests>

Produce. Open one pull request on your own repository, review your own difference, and note one thing you would have changed.

Day 4 • Break it deliberately • 45 minutes

Do. With your partner, create a merge conflict on purpose: both edit the same line on different branches, then merge. Resolve it properly.

Produce. A short note on what the three markers looked like and which line you kept. A conflict you have created on purpose is never frightening again.

Day 5 • Stop committing outputs • 45 minutes

Do. Install `nbstripout` in your repository as a Git filter. <https://github.com/kynan/nbstripout>

Produce. Commit a notebook after re running every cell, and paste the difference. If the filter is working, the outputs are absent from what Git recorded while still on your screen.

Day 6 • Audit your own repository • 45 minutes

Do. Run `git status` and read every untracked file. For each, decide: written by a human, or produced by a machine? Write the ignore file accordingly.

Produce. The committed `.gitignore`, and the before and after status output. This is assignment Set 3, Q3.

Day 7 • Connect it to the model • 60 minutes

Do. Train your model, then record the commit hash beside it. `git rev-parse -short HEAD` prints it. Put it in the run record you started in Lecture 04.

Why this matters. You have just built, by hand, the lineage link that the tracking server will create automatically from Lecture 11. Doing it manually once is what makes the automatic version comprehensible rather than magical.

If you only have one hour all week

Day 6, then Day 7. Knowing what is in your repository and why, and connecting a model to the commit that made it. Everything else can be learned as you need it.

PART VI

Resource Library

All links verified on 9 September 2026.

25 For this lecture specifically

Resource	Where, and why
Chacon and Straub, <i>Pro Git</i> , second edition, free	https://git-scm.com/book/en/v2 Chapters 1 to 3 cover everything in this lecture, in more depth and better than any tutorial
GitHub Docs, about branches	https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-branches
GitHub Docs, about pull requests	https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests
nbstripout	https://github.com/kynan/nbstripout The Git filter that keeps notebook outputs out of the repository while leaving your working copy intact
GitHub Docs, personal access tokens	https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens Read the warning about treating tokens as passwords, in light of Section 18

26 Carried forward

Resource	Where
Sato, Wider and Windheuser, Continuous Delivery for Machine Learning	https://martinfowler.com/articles/cd4ml.html The three axes of change: code, model and data. Today was the first
Sculley et al., Hidden Technical Debt in Machine Learning Systems	https://proceedings.neurips.cc/paper_files/paper/2015/file/86df7dcfd896fc2674f757a2463eba-Paper.pdf
Amershi et al., Software Engineering for Machine Learning, ICSE 2019	https://www.microsoft.com/en-us/research/wp-content/uploads/2019/03/amershi-icse-2019_Software_Engineering_for_Machine_Learning.pdf

Resource	Where
scikit-learn, common pitfalls and recommended practices	https://scikit-learn.org/stable/common_pitfalls.html
Google, Rules of Machine Learning	https://developers.google.com/machine-learning/guides/rules-of-ml
Machine Learning in Production, DeepLearning.AI, free to audit	https://www.coursera.org/learn/introduction-to-machine-learning-in-production
MLOps Zoomcamp, DataTalks.Club	https://github.com/DataTalksClub/mlops-zoomcamp
Made With ML	https://madewithml.com
Stanford CS 329S	https://stanford-cs329s.github.io

If it is not committed, it does not exist.

Next lecture. Lecture 06, data versioning. Git remembers every version of our code; ask it to remember a two gigabyte dataset and it chokes. Why Git alone fails for large files, how DVC keeps the pointer in Git while the file lives elsewhere, and switching the whole project between two dataset versions. Bring your repository, because we add DVC to it.